

CIT4407 Assignment 2 2026

Specifications

Submission deadline: 11:59 pm, Friday 22 May 2026.

Value: 20% of CITS4407.

To be done individually.

This assignment mirrors some of the steps required for any data-science analysis. In particular, you will be implementing a data-cleaning and an analysis stage. You will be creating two Shell scripts, which will use Unix tools, e.g. sed, awk, and/or call other Shell scripts. The top level scripts are to be called **clean** and **analyse** (Note: no suffix!). Please make sure you use the specified names, as those are the names which the testing software will use to test your script. The input data, will be two CSV (comma separated value) files named:

- For question 1: **trending_videos_unclean.csv**
- For question 2: **trending_videos_clean.csv**

Submission Process

You are required to submit exactly one file. The file you submit must be a zip file. Your zip file must be named after your student number and have the .zip extension (e.g. if your student number is **12345678**, your zip file should be named **12345678.zip**). Your zip file must contain the following four files:

- A file named **clean** for data cleaning, completed in question 1,
- A file named **analyse** for data analysis, completed in question 2,
- A document named **prompts.pdf**, which contains a log of the AI prompts used to complete the assignment. You may use as many prompts as you like; however, the document should only include a maximum of 20 of your most frequently used prompts.
- A copy of your **.git** folder, renamed as **git_backup**, which contains all data relevant to Git. After completing the assignment, and while in your project directory, you can generate this folder from the hidden **.git** folder as follows: **cp -r .git git_backup**

You must submit this zip file by uploading it to the assignment page.

Can I use AI tools to complete my assignment?

This assignment is to be completed with limited use of AI. This means that the majority of the work should be completed by you, and AI tools can be used once you feel you have made a substantial contribution, to assist you in completing the assignment. The following are the two cases in which you are permitted to use AI tools:

1. Syntax corrections in your code: You should write the outline of your Bash scripts, the intended logical steps and Linux commands/tools that you plan to implement to produce the specified output. AI tools can then be used to help with syntax corrections in your code.
2. Grammar and spelling in comments: After your code is complete, functional and includes appropriate comments, AI may be used to support the grammar and spelling for those code comments.

So come up with your own logic/structure/algorithm, and use AI for syntax/grammatical corrections, and not for the logic itself. Make good use of this assignment to strengthen your knowledge of the Linux commands/tools. Use your own understanding to scrutinise any AI-generated code, and ensure that the final code you submit, is something that you understand completely!

Question 1: Error Checks & Data Cleaning

Data

Similar to Assignment 1, the data that you will be using for Q1 has been taken from [Kaggle](#). The downloaded data, on [Trending YouTube Video Statistics and Comments](#), includes data gathered from videos on YouTube that are contained within the trending category each day, from 2008 to 2018. The data, `trending_videos_unclean.csv`, has undergone a little pre-processing to make it suitable for the tasks in this assignment.

Have a look at the structure of the data to ensure you understand it. Each element of the row is separated by a comma, giving a total of 7 columns:

- *video_id*: The ID of the trending YouTube video.
- *publish_date*: The date and time when the video was published.
- *views*: The total number of views the video received.
- *likes*: The total number of likes the video received.
- *dislikes*: The total number of dislikes the video received.
- *comments_disabled*: Indicates whether comments are disabled for the video.
- *ratings_disabled*: Indicates whether ratings are disabled for the video.

Task

You are to create a top level Bash script called `clean`, which reads data from `trending_videos_unclean.csv`, which will be placed in the same directory as your script. The input data contains a few errors and missing values. Your task is to write a script that performs the following error-checks and data cleaning:

Error checks:

Error Case	Output to print
1) No input file specified	ERROR: No input CSV file provided
2) Input file not found in the current directory	ERROR: Input file not found in the current directory
3) Input file is not a CSV file	ERROR: Input file expected in a CSV format
4) Input file is empty	ERROR: Empty file provided
5) Input CSV file has incorrect number of columns	ERROR: Expected 7 columns in the input

After error-checking, it is important to “clean” the data. This means removing/fixing any corrupt or incorrect data. Your script must perform the following data-cleaning operations:

- Delete the column named, *ratings_disabled*, as it provides redundant information.
- Delete rows that do not contain the same number of cells as the header.
- Delete any duplicate rows from your data.
- Delete rows that do not contain a *video_id*.
- Delete rows which contain zero number of likes or dislikes.
- Currently, the *publish_date* column also has the time in it. Remove the time data from the *publish_date* column. For example:
2009-09-18T15:36:33.000Z should be cleaned to look like: 2009-09-18

You can assume that the order of the columns in the input data will not change.

Sample Output

Your script should read your input data as a command-line argument as follows:

```
./clean trending_videos_unclean.csv
```

The cleaned file, named: `trending_videos_clean.csv`, is uploaded on LMS for reference, and can be found in the Assignment 2 folder.

Question 2: Data Analysis

Data

The input data for this task is the cleaned version of the `trending_videos_unclean.csv` data, named: `trending_videos_clean.csv`

Task

Once your data has been cleaned, it can be analysed to gain some insights! Write a script, called `analyse`, which performs the following tasks:

- Calculate and print the *video_id* of the video that has the most number of occurrences in the data.
- Calculate and print the mean (2 places after the decimal) number of views.
- Calculate and print the *video_id* of the video with the maximum number of dislikes.
- Print the *video_id* and the *publish_date* of the video that contains the highest engagement rate, which is an indicator of how actively the viewers react. The engagement rate is calculated as: $(\text{likes} + \text{dislikes})/\text{views}$.
- Print the *video_id* and the *publish_date* of the video that contains the least net sentiment rate, which determines if the overall reception is positive or negative. The net sentiment rate is calculated as: $(\text{likes} - \text{dislikes})/\text{views}$.

If there is a tie between two or more videos, print the required data for all of them in a clear and sensible format.

Sample Output

Your script should read your input data as a command-line argument as follows:

```
./analyse trending_videos_clean.csv
```

Your script should direct the output to standard output in the following format:

Most frequent video, ID: id4667

Mean number of views: 2355595.97

Max dislikes video, ID: id2798

Highest engagement rate video, ID: id2282, dated: 2018-01-04

Least sentiment rate video, ID: id2219 , dated: 2017-12-13

Marking

This assignment is worth a total of 20 marks. The marks are distributed as follows:

- 1) Correctness Testing (14 marks): This process will be largely automated, and the number of points you score depends on how many test-cases, both seen and unseen, your script is able to complete.
- 2) Style/Maintainability (4 marks): Programs are written as much for humans as for computers. Hence, it is important that your code is readable and maintainable. This includes comments, meaningful variable names for significant variables, and sensible anti-bugging. It also includes making sure your program removes any temporary files that were created along the way! For style/maintainability, the rubric is:
 - $0 \leq x < 1$ Gibberish, impossible to understand
 - $1 \leq x < 2$ Style is poor, but can see where the train of thoughts is heading
 - $2 \leq x < 3$ Style is acceptable with some lapses
 - $3 \leq x < 4$ Style is good or very good, with small lapses
 - 4 Excellent style, really easy to read and follow
- 3) Using Git (2 marks): You must have multiple commits at different times with legible commit messages. One commit at the start, and one at the end of the project, is not sufficient! For using Git, the rubric is:
 - $0 \leq x < 1$ No use of Git
 - $1 \leq x < 2$ Git is used, but not informative or just over one day
 - 2 Multiple commits over time with relevant commit messages

Important things to Note

1. You are free and encouraged to use any Linux commands/tools covered in this unit from Week 1 to Week 11.
2. Automated testing is being used so that all submitted programs are being tested the same way. Sometimes it happens that there is one mistake in a program that means that no tests are passed. If the human marker is able to spot the cause and fix it readily, then they are allowed to do that, and your - now fixed - program will score whatever it scores from the tests, minus 2 marks, because other students will not have had the benefit of marker intervention. That's way better than getting zero for the run-time tests, right? (On the other hand, if the bug is too hard to fix, the marker needs to move on to other submissions.)
3. **Most Important:** It's okay to develop your code on some other version of Linux, but make sure you test the final version, prior to submission, using the class Docker image.